Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster Universitario en Ingeniería Informática

Simulador web de escenarios de movilidad urbana mediante técnicas de Inteligencia Artificial

Iván Vicente Hernández García de Mora

Noviembre, 2025

Escuela Superior de Ingeniería Informática Albacete

Escuela Superior de Informática

TRABAJO FIN DE MÁSTER

Máster Universitario en Ingeniería Informática

Simulador web de escenarios de movilidad urbana mediante técnicas de Inteligencia Artificial

Autor: Iván Vicente Hernández García de Mora

Director: Nombre del director

Codirector: Nombre del codirector

UCLM Universidad Castilla-La Mancha

Noviembre, 2025

Aquí va la dedicatoria que cada cual quiera escribir. El ancho se controla manualmente

Declaración de autoría

Yo, … , con DNI … , declaro que soy el único autor del trabajo fin de grado titulado “ … ”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Albacete, a … de … de 20 …

Fdo.: Iván Vicente Hernández García de Mora

Resumen

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Agradecimientos

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Índice general

1 Introducción 1

1.1 Contexto y motivación 1

1.2 Objetivos del TFM 1

1.2.1 Objetivo general 1

1.2.2 Objetivos específicos 1

1.2.3 Competencias trabajadas 2

1.3 Alcance y limitaciones 2

2 Estado del arte y marco tecnológico 3

2.1 Fundamentos y enfoques 3

2.2 Datos abiertos para movilidad 3

2.2.1 OpenStreetMap 4

2.2.2 GTFS 5

2.3 Tecnologías abiertas de enrutado 6

2.3.1 OSRM 6

2.3.2 OpenTripPlanner 7

2.4 Plataformas propietarias y servicios gestionados 7

2.4.1 Google Maps Platform 8

2.4.2 Otras plataformas de referencia 8

2.5 Modelado de elección modal 9

2.5.1 Modelos de utilidad aleatoria 10

2.5.2 Modelo Logit Multinomial 11

2.5.3 Limitaciones del enfoque clásico 13

2.5.4 Aprendizaje automático para elección modal 13

2.5.5 Random Forests 14

2.5.6 Gradient Boosting Decision Trees *15*
2.5.7 Redes Neuronales Profundas *16*

3 Metodología **17**

3.1 SCRUM adaptado a un desarrollo unipersonal 17

3.2 Product Backlog 19

3.2.1 Backlog inicial. *19*
3.2.2 Priorización de items *20*

3.3 Histórico de sprints ejecutados 20

3.3.1 Sprint 1: Prototipo OSRM remoto *21*
3.3.2 Sprint 2: OSRM local multi-perfil *21*
3.3.3 Sprint 3: GTFS Toledo. *22*
3.3.4 Sprint 4: OTP multimodal. *22*
3.3.5 Sprint 5: Mejora visual y UX *22*
3.3.6 Sprint 6: LPMC exploración y preprocesado. *23*
3.3.7 Sprint 7: XGBoost baseline *23*
3.3.8 Sprint 8: Arranque de memoria y trazabilidad SCRUM *23*
3.3.9 Sprint 9: Integración de inferencia LPMC en simulador *24*

3.4 Sprint reviews y retrospectivas 24

3.4.1 Aprendizajes por iteración *24*
3.4.2 Acciones de mejora *25*

3.5 Seguimiento operativo 25

3.5.1 Registro en SCRUM_LOG *25*
3.5.2 Trazabilidad con el repositorio *25*

3.6 Próximo sprint propuesto 26

3.6.1 Objetivo *26*
3.6.2 Alcance y criterio de cierre *26*
3.6.3 Plan a dos sprints vista *26*

4 Arquitectura/Diseño de la solución **27**

4.1 Arquitectura general del sistema 27

4.2 Diseño del backend 27

4.3 Diseño del frontend 27

4.4 Integración OSRM, OTP y GTFS 27

4.5 Diseño de integración del modelo IA 27

4.6 Decisiones técnicas y trade-offs 27

5 Implementación y resultados **29**
5.1 Implementación de OSRM local. 29
5.1.1 Preprocesado por perfiles *29*
5.1.2 Despliegue de contenedores *29*
5.2 Implementación de OTP con Toledo 29
5.2.1 Construcción de grafo *29*
5.2.2 Explotación del planificador. *29*
5.3 Implementación del backend. 29
5.3.1 Endpoints y servicios *29*
5.4 Implementación del frontend 30
5.4.1 Interfaz y mapa interactivo *30*
5.5 Integración de datos GTFS 30
5.5.1 Carga y procesamiento *30*
5.6 Entrenamiento de los modelos de aprendizaje automático 30
5.6.1 Descripción del dataset *30*
5.6.2 Preprocesado de datos *30*
5.6.3 Ajuste de hiperparámetros *30*
5.6.4 Entrenamiento y evaluación *30*

6 Conclusiones y trabajo futuro. **31**
6.1 Conclusiones técnicas 31
6.2 Cumplimiento de objetivos 31
6.3 Trabajo futuro 31

A Anexos. **33**
A.1 Manual de despliegue y ejecución 33
A.1.1 Requisitos del entorno. *33*
A.1.2 Arranque de servicios *33*
A.2 Endpoints y contratos API 33
A.2.1 Contratos del backend. *33*
A.3 Tablas de hiperparámetros y resultados completos 33
A.3.1 Configuraciones experimentales. *33*
A.4 Evidencias de sprints 33
A.4.1 Registro de tareas y decisiones *34*

Referencia bibliografica. **38**

Índice de figuras

2.1 Esquema conceptual del flujo de información en un simulador de movilidad como el desarrollado en este TFM. Elaboración propia. 4
2.2 Forma sigmoidal ilustrativa de la probabilidad de elección en el modelo MNL. Elaboración propia a partir de [Martín-Baos, 2023, Train, 2009, p. 16]. . . . 12

Índice de tablas

2.1 Comparación resumida de las soluciones de enrutado consideradas en este trabajo. Elaboración propia a partir de [Project OSRM, 2026a, OpenTripPlanner Team, 2026b, Google, 2026b, Google, 2026a]. 9

Índice de algoritmos

Índice de listados de código

1. Introducción

1.1. Contexto y motivación

1.2. Objetivos del TFM

1.2.1. Objetivo general

Desarrollar un prototipo de simulador web de movilidad urbana que integre modelos de predicción basados en Machine Learning y servicios de enrutado, con el fin de analizar el impacto de distintas políticas de transporte en el reparto modal de los viajes y generar métricas de apoyo para la planificación y la toma de decisiones.

1.2.2. Objetivos específicos

- Entrenar y validar modelos de Machine Learning con datos de movilidad para predecir la elección modal de los usuarios en función de variables sociodemográficas, de tiempo y de coste.
- Integrar servicios de enrutado (OSRM y OpenTripPlanner) para obtener tiempos y costes realistas entre pares origen-destino en el entorno urbano.
- Desarrollar una aplicación web interactiva que permita definir viajes sobre un mapa, simular el comportamiento de los usuarios y visualizar resultados en un cuadro de mando.
- Implementar un módulo de análisis de escenarios *what-if* para modificar parámetros de política (coste del coche, tarifas del transporte público, frecuencia de autobuses, etc.) y observar su efecto en el reparto modal.

- Generar métricas de impacto (reparto modal, tiempo medio de viaje y estimación de emisiones de CO_2) como apoyo a la toma de decisiones en movilidad urbana.

1.2.3. Competencias trabajadas

El desarrollo del TFM contribuye a las siguientes competencias del Máster:

- **CP01**: Integrar tecnologías, aplicaciones, servicios y sistemas de la Ingeniería Informática en un contexto multidisciplinar.
- **CP05**: Aplicar métodos matemáticos, estadísticos y de inteligencia artificial para modelar y desarrollar sistemas inteligentes.
- **CP07**: Realizar un proyecto integral original de carácter profesional que sintetice las competencias adquiridas en el plan de estudios.

1.3. Alcance y limitaciones

2. Estado del arte y marco tecnológico

2.1. Fundamentos y enfoques

La movilidad urbana se ha consolidado como un ámbito de estudio en el que confluyen la ingeniería del transporte, la analítica de datos y la inteligencia artificial. En este contexto, los simuladores actuales no se limitan a representar mapas o a calcular rutas aisladas, sino que actúan como herramientas para evaluar políticas y escenarios de movilidad, permitiendo analizar de forma anticipada cómo cambios en los costes, en la oferta de transporte o en las condiciones operativas pueden modificar el comportamiento de la población.

Este tipo de sistemas exige combinar varias capas de conocimiento. Por un lado, la capa de datos, que determina la calidad y la trazabilidad de la información sobre la red de transporte y los servicios de transporte público asociados. Por otro, la capa de enrutado, que permite transformar un par origen-destino en variables de servicio observables (tiempo, distancia, segmentos de viaje, transbordos o coste aproximado). Finalmente, la capa de modelado, en la que dichas variables se integran con atributos sociodemográficos para estimar la elección modal y construir escenarios *what-if*. Esta arquitectura en capas es coherente con la evolución reciente de herramientas de simulación en investigación y planificación [Horni et al., 2016, Lopez et al., 2018].

2.2. Datos abiertos para movilidad

El desarrollo de los sistemas descritos en el apartado anterior requiere la disponibilidad de grandes volúmenes de datos fiables y reutilizables. Por este motivo, en los últimos años han adquirido gran protagonismo los conjuntos de datos abiertos en movilidad, ya que

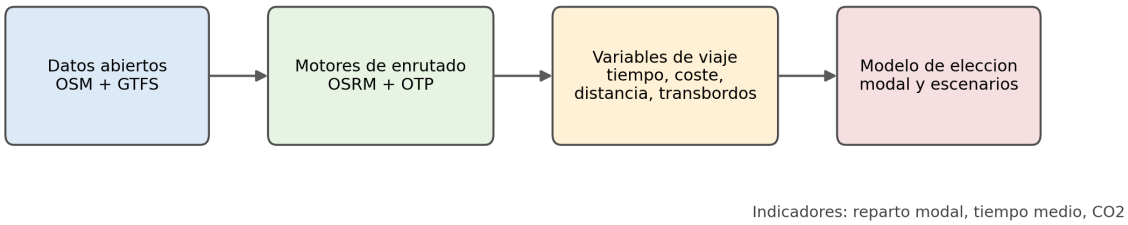

Figura 2.1: Esquema conceptual del flujo de información en un simulador de movilidad como el desarrollado en este TFM. Elaboración propia.

permiten construir, validar y reproducir modelos de análisis sin depender de fuentes propietarias cerradas. En este contexto, OpenStreetMap y GTFS se han consolidado como los dos pilares más utilizados para representar, respectivamente, la red urbana y la oferta de transporte público.

2.2.1. OpenStreetMap

OpenStreetMap (OSM) es una base de datos geográfica abierta y colaborativa ampliamente utilizada en proyectos de movilidad, tanto en investigación como en aplicaciones operativas. La propia fundación del proyecto define OSM como una fuente de datos abiertos reutilizable por terceros, con cobertura global y actualización continua por parte de la comunidad [OSM Foundation, 2026]. Esta naturaleza abierta resulta especialmente relevante en un trabajo experimental, ya que permite reproducir resultados a partir de un conjunto de datos público, versionable e independiente de condiciones comerciales impuestas por terceros.

En términos de calidad cartográfica, diversos estudios han mostrado que OSM puede ofrecer niveles de precisión adecuados para el análisis del transporte, aunque con cierta heterogeneidad espacial entre zonas y entre tipos de elemento, por ejemplo, entre la red viaria principal y los detalles de la infraestructura peatonal [Haklay, 2010]. Por ello, en este trabajo OSM se considera una fuente adecuada para construir la red de enrutado, teniendo en cuenta las características específicas del área de estudio al interpretar los resultados.

2.2.2. GTFS

GTFS (*General Transit Feed Specification*) es el estándar abierto dominante para publicar oferta de transporte público programada. La documentación oficial describe de forma precisa el contrato de datos entre productor y consumidor: estructura de ficheros, relaciones entre entidades y restricciones semánticas necesarias para que un planificador pueda reconstruir correctamente servicios, horarios y calendarios [MobilityData, 2026b]. En otras palabras, GTFS no es solo un formato de intercambio, sino también un marco común de interoperabilidad entre operadores, agregadores y motores de planificación.

La estructura del estándar se basa en un conjunto de ficheros fundamentales, entre los que destacan `stops.txt`, `routes.txt`, `trips.txt`, `stop_times.txt`, `calendar.txt` y `calendar_dates.txt`. Estos archivos permiten representar simultáneamente la topología de la red y la dimensión temporal del servicio [MobilityData, 2026b]. Esta separación resulta esencial, ya que el componente espacial define dónde se presta el servicio y el componente temporal determina cuándo se presta. En un entorno de simulación multimodal, ambos planos deben mantenerse coherentes para evitar itinerarios inviables o resultados inconsistentes.

En el contexto español, el canal institucional más relevante para localizar y descargar feeds GTFS es el *Punto de Acceso Nacional de Transporte Multimodal* (NAP), gestionado por el Ministerio de Transportes y Movilidad Sostenible. La propia descripción oficial del portal lo define como el punto único nacional para concentrar información de oferta de transporte de viajeros, en línea con el marco europeo de datos de movilidad [NAP Transporte Multimodal, 2026a, NAP Transporte Multimodal, 2026b].

En este TFM, la descarga de datos para transporte urbano se realizó precisamente desde NAP. Como ejemplo de referencia nacional se consultó el conjunto de EMT Valencia y, para el caso de estudio del proyecto, el conjunto de autobuses urbanos de Toledo [EMT Valencia, 2026, Unauto, 2026]. En ambos casos, la página de detalle expone metadatos operativos (por ejemplo, número de paradas y formatos de descarga), así como los atributos incluidos en el conjunto de datos, como la accesibilidad, las tarifas o la geometría de las rutas. Esta información permite verificar de forma rápida el tamaño y la estructura del feed antes de integrarlo en el flujo de procesamiento del simulador.

La propia comunidad GTFS publica recomendaciones de calidad de publicación (persistencia de identificadores, estabilidad de URL de feed, cobertura temporal mínima y buenas prácticas de consistencia), que son relevantes para garantizar explotación automatizada en entornos de producción o experimentación [MobilityData, 2026a]. Además, el validador canónico mantenido por MobilityData se ha convertido en herramienta estándar para control

de calidad de feeds estáticos antes de su uso analítico [MobilityData, 2026c]. En este trabajo, GTFS cumple una doble función: por un lado, alimentar la capa de consulta de la red de autobús urbano y, por otro, proporcionar los datos necesarios para construir el grafo multimodal empleado posteriormente por OpenTripPlanner.

2.3. Tecnologías abiertas de enrutado

Una vez descritas las fuentes de datos que permiten representar la red urbana y la oferta de transporte público, la segunda pieza fundamental de un simulador de movilidad es el algoritmo de enrutado. Estos motores permiten transformar la información geoespacial y temporal en itinerarios concretos, así como obtener variables de servicio relevantes, como el tiempo de viaje, la distancia recorrida, los transbordos o las fases de acceso y egreso. En este trabajo se emplean dos tecnologías abiertas con funciones complementarias: OSRM, orientado al cálculo eficiente de rutas sobre red viaria, y OpenTripPlanner, especializado en planificación multimodal con transporte público.

2.3.1. OSRM

OSRM (*Open Source Routing Machine*) es uno de los motores abiertos más utilizados para cálculo de rutas sobre red viaria OSM, especialmente cuando se necesitan tiempos de respuesta bajos y gran volumen de consultas [Project OSRM, 2026a, Project OSRM, 2026b]. Su interés en un simulador de movilidad no está solo en obtener una ruta entre dos puntos, sino en poder generar de forma sistemática atributos de viaje (tiempo, distancia o matrices origen-destino) que después se integran en análisis y modelos predictivos.

La principal fortaleza técnica de OSRM es su enfoque de preprocesado: antes de servir consultas, el sistema transforma el extracto `.osm.pbf`, que es el formato binario comprimido habitualmente utilizado para distribuir datos de OpenStreetMap, en estructuras optimizadas de búsqueda. Este diseño desplaza coste al inicio, pero permite consultas rápidas y estables durante la fase de explotación. Además, OSRM trabaja con perfiles de movilidad diferenciados (por ejemplo, coche, bicicleta y a pie), lo que permite modelar de forma explícita costes de viaje distintos por modo. A nivel interno, OSRM se apoya en algoritmos de aceleración de rutas: MLD (*Multi-Level Dijkstra*), que divide la red en niveles para reducir el espacio de búsqueda en cada consulta, y CH (*Contraction Hierarchies*), que simplifica el grafo con atajos para acelerar rutas largas. [Delling et al., 2017, Geisberger et al., 2008].

2.3.2. OpenTripPlanner

OpenTripPlanner (OTP) es un planificador multimodal diseñado para combinar red de calle (OSM) y red de transporte público programado (GTFS) en un único problema de viaje puerta a puerta [OpenTripPlanner Team, 2026b, OpenTripPlanner Team, 2026c]. Su función difiere de la de un motor de enrutado viario convencional, ya que no se limita a calcular el tramo sobre la red de calles, sino que también representa de manera integrada las fases de acceso peatonal, los tiempos de espera, el recorrido en transporte público, los posibles transbordos y el tramo final hasta el destino.

La documentación oficial de OTP describe una arquitectura basada en construcción previa de grafo y posterior explotación por API de planificación. Ese grafo integra simultáneamente geometría de red y programación temporal del servicio, por lo que la consulta de rutas depende de fecha y hora, no solo de coordenadas [OpenTripPlanner Team, 2026a, OpenTripPlanner Team, 2026d]. Esta dimensión temporal es la base para representar transporte público de forma operativa.

En la salida, OTP devuelve itinerarios descompuestos en tramos o segmentos de viaje, denominados *legs* en su propia documentación, con información sobre el modo utilizado, los tiempos parciales, el número de transbordos y los metadatos de cada línea. Esta estructura resulta especialmente útil para el análisis multimodal, ya que permite distinguir con precisión las distintas fases del desplazamiento. Por contraste, motores viarios como OSRM están orientados al cálculo rápido de rutas sobre red de calle y no incorporan de forma nativa la lógica propia del transporte público, con horarios, transbordos y secuencias diferenciadas dentro de un mismo itinerario.

2.4. Plataformas propietarias y servicios gestionados

Junto a las tecnologías abiertas de enrutado, el ecosistema actual de movilidad incluye también plataformas propietarias que ofrecen estos mismos servicios mediante API gestionadas. La relevancia industrial de este problema ha favorecido la aparición de soluciones comerciales muy maduras, orientadas tanto al desarrollo de producto como a la integración rápida en aplicaciones web y móviles. Aunque estas plataformas simplifican el despliegue técnico, también introducen condicionantes específicos en términos de coste, dependencia del proveedor y reproducibilidad.

2.4.1. Google Maps Platform

Google Maps Platform se ha convertido en un estándar de facto en los servicios comerciales de localización y navegación orientados al desarrollo de productos digitales. En particular, *Routes API* ofrece cálculo de rutas multimodo, metadatos de viaje y un ecosistema de integración especialmente maduro para aplicaciones web, móviles y, de forma destacada, entornos basados en Android [Google, 2026b].

Para proyectos de desarrollo de producto, este modelo reduce de forma notable la complejidad operativa, ya que no exige desplegar ni mantener infraestructura propia de enrutado y permite acceder a funcionalidades avanzadas mediante API. Esta característica ha favorecido su adopción en aplicaciones comerciales, especialmente en entornos móviles y ecosistemas integrados con servicios de Google. No obstante, en contextos de investigación aplicada o en escenarios con un volumen elevado de simulaciones, aparecen limitaciones relevantes: por un lado, el coste económico crece con el número de peticiones y con el tipo de servicio consumido; por otro, la ejecución queda condicionada por cuotas de uso, políticas del proveedor y disponibilidad externa de la plataforma [Google, 2026a]. Estas restricciones reducen su idoneidad cuando se busca trazabilidad completa, control experimental y escalabilidad económica predecible.

2.4.2. Otras plataformas de referencia

Además de Google Maps Platform, el ecosistema actual incluye otras alternativas consolidadas de consumo por API, entre ellas Mapbox [Mapbox, 2026] o TomTom [TomTom, 2026]. Estas plataformas ofrecen catálogos amplios (rutas, matrices, navegación, tráfico y servicios avanzados según proveedor), y comparten una lógica común de uso gestionado mediante credenciales, límites de consumo y planes de servicio.

También existen servicios intermedios como openrouteservice, que proporcionan API pública y, al mismo tiempo, opciones de despliegue propio [HeiGIT gGmbH, 2026, openrouteservice, 2026]. Este tipo de solución puede ser útil como puente entre experimentación rápida y control de infraestructura, aunque con sus propias restricciones de uso en modalidad pública.

Esta comparación resume la decisión tecnológica adoptada en el proyecto. Para un simulador académico orientado a experimentación, trazabilidad y ejecución repetida de escenarios, la combinación OSRM + OTP en infraestructura propia ofrece un equilibrio más adecuado que una API comercial gestionada, aunque implique una mayor complejidad de despliegue.

Tabla 2.1: Comparación resumida de las soluciones de enrutado consideradas en este trabajo. Elaboración propia a partir de [Project OSRM, 2026a, OpenTripPlanner Team, 2026b, Google, 2026b, Google, 2026a].

Solución	Tipo de red	Despliegue	Coste de uso	Adecuación al TFM
OSRM	Red viaria OSM, perfiles por modo	Propio	Infraestructura propia	Muy adecuado para consultas masivas y control experimental
OTP	Red multimodal OSM + GTFS	Propio	Infraestructura propia	Muy adecuado para transporte público y análisis puerta a puerta
Google Maps Platform	API comercial multimodo gestionada	Tercero	Pago por petición y cuotas	Útil como referencia industrial, menos adecuado para reproducibilidad

2.5. Modelado de elección modal

Una vez descrita la infraestructura tecnológica que permite representar la red y calcular itinerarios, es necesario incorporar una capa de modelado que permita analizar el comportamiento de los viajeros. En un simulador de movilidad, no basta con conocer qué alternativas existen para un desplazamiento determinado, sino que también es necesario estimar cuál de ellas es más probable que sea elegida en función de sus atributos de servicio y de las características del usuario. Esta cuestión se aborda habitualmente mediante modelos de elección discreta y, más recientemente, mediante técnicas de aprendizaje automático aplicadas a la predicción de la elección modal [Train, 2009, Martín-Baos, 2023].

En la literatura reciente, una de las referencias más utilizadas para este tipo de problemas es el dataset London Passenger Mode Choice (LPMC), construido a partir de viajes observados en Londres y enriquecido con variables de nivel de servicio para distintos modos

de transporte [Hillel et al., 2018, Hillel, 2019]. Su interés en este trabajo no está en el caso de estudio concreto, sino en que proporciona una base metodológica y experimental muy próxima al problema abordado en este TFM.

2.5.1. Modelos de utilidad aleatoria

Este problema se ha abordado tradicionalmente mediante los denominados modelos de utilidad aleatoria (*Random Utility Models*, RUM), ampliamente utilizados en transporte para analizar decisiones de elección discreta, como la elección del modo de transporte [Martín-Baos, 2023, p. 14]. En este enfoque, el decisor elige una alternativa dentro de un conjunto de opciones mutuamente excluyentes, por ejemplo caminar, usar la bicicleta, utilizar transporte público o desplazarse en coche [Train, 2009, McFadden, 1974].

La idea central de estos modelos es que cada viajero asocia una utilidad a cada alternativa disponible dentro de su conjunto de elección, y se asume que elegirá aquella que le proporcione una mayor utilidad. Sin embargo, dicha utilidad no puede observarse de forma completa, ya que parte de los factores que influyen en la decisión sí pueden ser medidos por el analista, mientras que otros permanecen ocultos o no observados. Por ello, la utilidad de una alternativa suele descomponerse en un término determinista y un término aleatorio [Martín-Baos, 2023, Train, 2009, p. 14]:

$$U_{in} = V_{in} + \varepsilon_{in} \tag{2.1}$$

donde U_{in} representa la utilidad total que el individuo n asocia a la alternativa i, V_{in} es la componente sistemática o determinista de dicha utilidad, y ε_{in} recoge los factores no observados que también influyen en la decisión.

Bajo este planteamiento, el viajero se considera racional en el sentido de que selecciona la alternativa que maximiza su utilidad. En consecuencia, la elección observada puede expresarse en términos probabilísticos como la probabilidad de que la utilidad de una alternativa sea mayor o igual que la del resto de alternativas disponibles [Martín-Baos, 2023, Train, 2009, p. 15]:

$$P_{in} = P(U_{in} \geq U_{jn} \, \forall j \in C) \tag{2.2}$$

donde C representa el conjunto de alternativas consideradas. De este modo, los RUM permiten traducir un problema de comportamiento individual en un modelo probabilístico de elección, capaz de relacionar atributos del viaje, características del individuo y probabilidad de seleccionar cada modo de transporte.

2.5.2. Modelo Logit Multinomial

Entre los modelos derivados del enfoque RUM, el más extendido en el análisis de elección modal es el Modelo Logit Multinomial (*Multinomial Logit*, MNL). Su popularidad se debe a que ofrece una formulación matemática sencilla, una interpretación clara de los parámetros y una estimación relativamente eficiente desde el punto de vista computacional. En el contexto del transporte, el MNL ha sido utilizado de forma recurrente para estudiar cómo variables como el tiempo de viaje, el coste, los transbordos o determinadas características sociodemográficas influyen en la probabilidad de elegir un modo de transporte frente a otro [McFadden, 1974, Train, 2009, Martín-Baos, 2023].

El modelo MNL se obtiene al asumir que el término aleatorio de la utilidad sigue una distribución Gumbel independiente e idénticamente distribuida para todas las alternativas. Bajo esta hipótesis, la probabilidad de que el individuo n elija la alternativa i puede expresarse de la siguiente forma [Martín-Baos, 2023, McFadden, 1974, Train, 2009, p. 15]:

$$P_{in} = \frac{\exp(V_{in})}{\sum_{j=1}^{I} \exp(V_{jn})} \tag{2.3}$$

donde I es el número total de alternativas disponibles y V_{in} representa la utilidad sistemática de la alternativa i para el individuo n. Esta expresión da lugar a una función de probabilidad de tipo sigmoidal, de manera que pequeñas variaciones en la utilidad representativa tienen mayor efecto cuando la probabilidad de elección se encuentra en valores intermedios que cuando una alternativa ya es claramente dominante o claramente poco atractiva. Esta propiedad resulta especialmente útil para interpretar cambios marginales en los atributos del servicio, por ejemplo ante mejoras en tiempo o coste de una determinada alternativa [Martín-Baos, 2023, Train, 2009, p. 15].

En la práctica, la componente determinista de la utilidad suele especificarse como una combinación lineal de atributos observables:

$$V_{in} = \beta_{i0} + \sum_{k} \beta_{ik} x_{ink} \tag{2.4}$$

donde x_{ink} representa el valor del atributo k asociado a la alternativa i para el individuo n, β_{ik} es el parámetro que mide la influencia de dicho atributo y β_{i0} es una constante específica de la alternativa. Esta formulación permite cuantificar el efecto relativo de variables como el tiempo de viaje, el coste monetario, el número de transbordos o el acceso peatonal, entre otras. En términos interpretativos, los parámetros estimados indican cómo varía la utilidad de cada modo cuando cambian sus atributos observables [Train, 2009, Martín-Baos, 2023].

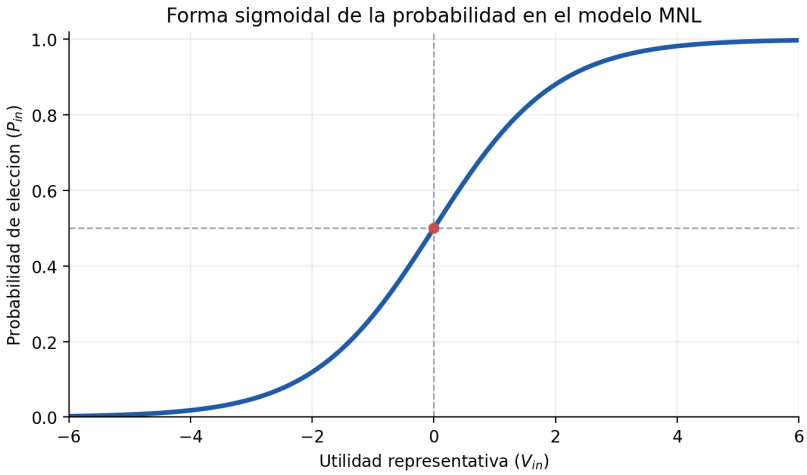

Figura 2.2: Forma sigmoidal ilustrativa de la probabilidad de elección en el modelo MNL. Elaboración propia a partir de [Martín-Baos, 2023, Train, 2009, p. 16].

La estimación de los parámetros del modelo se realiza habitualmente mediante máxima verosimilitud. Para ello, se define una función de verosimilitud que mide la probabilidad de observar las elecciones registradas en la muestra dadas unas determinadas estimaciones de los parámetros. El objetivo del proceso de ajuste consiste en encontrar los valores de β que maximizan dicha verosimilitud, o de forma equivalente, que maximizan la log-verosimilitud del modelo. Este procedimiento constituye la base clásica de estimación de modelos de elección discreta y permite obtener un modelo interpretable y coherente con la teoría del comportamiento del viajero [Train, 2009, Martín-Baos, 2023].

En este TFM, el modelo MNL resulta relevante como referencia metodológica, ya que proporciona una base teórica sólida para representar la elección modal a partir de atributos del desplazamiento. Además, sirve como punto de comparación frente a enfoques más flexibles basados en aprendizaje automático, que se describen en los apartados siguientes.

Una limitación conocida del modelo MNL es la hipótesis de independencia de alternativas irrelevantes (*Independence of Irrelevant Alternatives*, IIA), derivada de la suposición de errores independientes e idénticamente distribuidos. Esta propiedad implica que la relación de sustitución entre dos alternativas no depende de la presencia o de las características del resto de modos, lo cual puede resultar restrictivo en problemas de movilidad donde algunas opciones están más correlacionadas entre sí que otras [Martín-Baos, 2023, McFadden, 1974, Train, 2009, p. 15]. Aun así, el MNL sigue siendo el punto de partida más habitual en la literatura por su equilibrio entre simplicidad, interpretabilidad y capacidad descriptiva.

2.5.3. Limitaciones del enfoque clásico

A pesar de su amplia utilización en el análisis de elección modal, los modelos basados en utilidad aleatoria presentan ciertas limitaciones cuando se aplican a problemas complejos de movilidad. En primer lugar, requieren especificar de forma explícita la forma funcional de la utilidad sistemática, lo que implica que el analista debe decidir previamente qué variables incluir y cómo se relacionan con la utilidad de cada alternativa. En muchos casos, esta especificación se realiza mediante combinaciones lineales de atributos del viaje, lo que puede resultar restrictivo cuando las relaciones reales entre variables son no lineales o presentan interacciones complejas [Martín-Baos, 2023, p. 20].

Otra limitación relevante es que los modelos clásicos suelen asumir estructuras relativamente simples en la distribución de los términos de error. En el caso del modelo Logit Multinomial, por ejemplo, la hipótesis de independencia de alternativas irrelevantes puede no reflejar adecuadamente situaciones en las que algunas alternativas presentan correlaciones más fuertes entre sí, como ocurre frecuentemente entre distintos modos de transporte público o entre modos motorizados [Martín-Baos, 2023, p. 20].

Además, desde un punto de vista práctico, la especificación de modelos econométricos puede requerir un proceso iterativo de selección de variables, transformación de atributos y validación de supuestos teóricos. Aunque este enfoque proporciona modelos interpretables y coherentes con la teoría del comportamiento del viajero, puede resultar menos flexible cuando se trabaja con conjuntos de datos de alta dimensionalidad o con relaciones complejas entre variables [Martín-Baos, 2023, p. 20].

2.5.4. Aprendizaje automático para elección modal

Con el aumento de la disponibilidad de datos y de la capacidad computacional, en los últimos años han ganado relevancia los enfoques basados en aprendizaje automático para el análisis de elección modal. Desde esta perspectiva, la elección de modo puede formularse también como un problema de clasificación supervisada, en el que el objetivo consiste en predecir la alternativa elegida a partir de un conjunto de variables explicativas relacionadas con el viaje y con las características del individuo [Martín-Baos, 2023, p. 20].

A diferencia de los modelos clásicos de elección discreta, muchos algoritmos de aprendizaje automático no requieren especificar de forma explícita la forma funcional de la relación entre variables. En su lugar, aprenden patrones directamente a partir de los datos de entrenamiento, lo que les permite capturar relaciones no lineales, interacciones complejas entre atributos y estructuras de decisión difíciles de representar mediante modelos lineales tradi-

cionales [Martín-Baos, 2023, p. 21].

Entre las técnicas más utilizadas en este contexto destacan los métodos basados en árboles de decisión y sus variantes de ensamblado, como Random Forest o Gradient Boosting Decision Trees, así como modelos basados en redes neuronales profundas. Estos enfoques han mostrado un rendimiento predictivo competitivo en numerosos problemas de clasificación tabular, incluyendo aplicaciones en transporte y elección modal [José Ángel Martín-Baos, 2023, Martín-Baos, 2023].

En este trabajo se adopta esta perspectiva complementaria: el modelo Logit Multinomial se considera como referencia metodológica clásica, mientras que distintos modelos de aprendizaje automático se utilizan para explorar alternativas con mayor capacidad predictiva. En particular, la arquitectura del simulador se ha diseñado para permitir la integración de diferentes modelos de clasificación, entre ellos Random Forest, Gradient Boosting Decision Trees y redes neuronales profundas, de forma que puedan compararse sus resultados en el contexto del problema de elección modal analizado. Esta comparación resulta coherente con la evidencia reciente disponible para datasets reales de elección modal, entre ellos LPMC [José Ángel Martín-Baos, 2023, Hillel et al., 2018, Hillel, 2019].

2.5.5. Random Forests

Random Forest es un método de aprendizaje automático basado en el uso de múltiples árboles de decisión combinados mediante técnicas de ensamblado. El modelo fue propuesto por Breiman [Breiman, 2001] como una extensión de los árboles de clasificación tradicionales, con el objetivo de mejorar su estabilidad y capacidad predictiva. En la revisión de Martín Baos, este tipo de modelos aparece además como una de las familias más utilizadas en comparativas recientes de elección modal [Martín-Baos, 2023, p. 23].

La idea fundamental consiste en entrenar un gran número de árboles de decisión sobre diferentes subconjuntos de los datos de entrenamiento. Cada árbol se construye utilizando una muestra aleatoria del conjunto de datos original (técnica conocida como *bootstrap sampling*) y, además, en cada división del árbol se considera únicamente un subconjunto aleatorio de variables explicativas. Este doble proceso de aleatorización permite reducir la correlación entre árboles individuales y, en consecuencia, mejorar el rendimiento del modelo agregado [Breiman, 2001, Martín-Baos, 2023].

En problemas de clasificación, como el de elección modal, cada árbol produce una predicción independiente y la decisión final se obtiene mediante votación mayoritaria entre todos los árboles del bosque. Este mecanismo permite capturar relaciones no lineales entre variables y manejar de forma robusta conjuntos de datos con múltiples atributos

[Breiman, 2001].

En el ámbito del transporte, los modelos Random Forest se han utilizado en distintos trabajos para predecir la elección de modo de transporte, ya que ofrecen un buen equilibrio entre capacidad predictiva, robustez frente al sobreajuste y relativa facilidad de interpretación mediante medidas de importancia de variables. No obstante, la evidencia comparativa reciente sugiere que su rendimiento puede quedar por debajo del de otros métodos más potentes de boosting en algunos conjuntos de datos reales [José Ángel Martín-Baos, 2023].

2.5.6. Gradient Boosting Decision Trees

Los modelos basados en *Gradient Boosting Decision Trees* (GBDT) constituyen otra familia de métodos de aprendizaje automático ampliamente utilizados en problemas de clasificación con datos tabulares. A diferencia de los métodos basados en bagging, como Random Forest, el enfoque de boosting construye los árboles de decisión de forma secuencial, de modo que cada nuevo árbol intenta corregir los errores cometidos por el conjunto de árboles anteriores [Friedman, 2001, Martín-Baos, 2023].

El proceso comienza entrenando un primer árbol de decisión sencillo sobre los datos de entrenamiento. A continuación, se construyen árboles adicionales que se ajustan sobre los residuos o errores del modelo previo. De este modo, el modelo final se obtiene como la suma ponderada de múltiples árboles débiles, cada uno de los cuales contribuye a mejorar gradualmente la predicción global [Friedman, 2001].

Entre las implementaciones más conocidas de este enfoque se encuentra XGBoost (*Extreme Gradient Boosting*), una biblioteca optimizada que introduce mejoras en eficiencia computacional, regularización del modelo y manejo de grandes conjuntos de datos [Chen and Guestrin, 2016]. Gracias a estas características, XGBoost se ha convertido en una de las técnicas más utilizadas en problemas de aprendizaje supervisado con datos estructurados y ha mostrado un comportamiento especialmente competitivo en comparativas recientes de elección modal [José Ángel Martín-Baos, 2023].

En el contexto de este TFM, los modelos basados en Gradient Boosting resultan especialmente relevantes, ya que permiten capturar relaciones no lineales entre los atributos del viaje y la elección modal. Además, su buen rendimiento predictivo y su flexibilidad para manejar distintos tipos de variables los convierten en una herramienta adecuada para integrarse en el simulador de movilidad desarrollado en este trabajo.

2.5.7. Redes Neuronales Profundas

Las redes neuronales profundas (*Deep Neural Networks*, DNN) constituyen otra familia de modelos de aprendizaje automático que ha experimentado un crecimiento significativo en los últimos años. Estas técnicas se basan en arquitecturas formadas por múltiples capas de neuronas artificiales que transforman progresivamente las variables de entrada mediante combinaciones lineales y funciones de activación no lineales [Goodfellow et al., 2016, Martín-Baos, 2023].

A diferencia de los modelos basados en árboles, las redes neuronales permiten aprender representaciones internas de los datos a través de varias capas ocultas, lo que facilita capturar relaciones complejas entre variables. Esta capacidad de modelar interacciones no lineales ha motivado su aplicación en numerosos problemas de predicción y clasificación, incluyendo el análisis del comportamiento de los viajeros [Goodfellow et al., 2016].

En problemas de elección modal, las redes neuronales profundas pueden utilizarse para estimar la probabilidad de seleccionar cada modo de transporte a partir de atributos del desplazamiento y características sociodemográficas del usuario. Aunque estos modelos suelen requerir mayor volumen de datos y un proceso de ajuste más cuidadoso que los métodos basados en árboles, también ofrecen una elevada flexibilidad para capturar patrones complejos presentes en los datos [José Ángel Martín-Baos, 2023, Martín-Baos, 2023].

En este trabajo, las redes neuronales profundas se consideran como una posible extensión del sistema de predicción implementado en el simulador de movilidad. La arquitectura del sistema permite integrar distintos modelos de clasificación, lo que abre la posibilidad de incorporar en futuras versiones modelos neuronales entrenados sobre los mismos atributos de viaje utilizados por los métodos basados en árboles.

3. Metodología

Este capítulo recoge la gestión del proyecto con un enfoque SCRUM adaptado a un TFM individual. Se busca dejar trazabilidad clara de qué se planificó, qué se ejecutó, qué bloqueos aparecieron y qué decisiones técnicas se tomaron en cada iteración.

3.1. SCRUM adaptado a un desarrollo unipersonal

SCRUM es una metodología ágil de carácter iterativo e incremental, orientada a organizar el trabajo en ciclos cortos, priorizar entregas de valor y revisar de forma continua el avance del producto [Schwaber and Sutherland, 2020]. Aunque su formulación original está pensada para equipos, sus principios también pueden adaptarse a proyectos individuales cuando se busca mantener planificación, trazabilidad y capacidad de reacción ante cambios.

En este trabajo se ha utilizado una adaptación práctica de SCRUM, no como aplicación estricta del marco en todos sus elementos formales, sino como guía para estructurar el desarrollo del simulador, registrar decisiones y ordenar la evolución del proyecto. Esta adaptación ha resultado útil por tres motivos. En primer lugar, porque se han combinado tareas heterogéneas, incluyendo desarrollo backend, frontend, integración de motores de enrutado, tratamiento de datos, entrenamiento de modelos y redacción de memoria. En segundo lugar, porque el alcance real del proyecto ha ido evolucionando a medida que se validaban soluciones técnicas y se detectaban bloqueos. Por último, porque el seguimiento periódico con el tutor ha funcionado como mecanismo natural de revisión y reorientación.

Desde el punto de vista organizativo, la Guía Scrum 2020 define un *Scrum Team* compuesto por tres responsabilidades principales: *Product Owner*, *Scrum Master* y *Developers* [Schwaber and Sutherland, 2020]. En un trabajo unipersonal esta separación no puede reproducirse de forma literal, por lo que se ha reinterpretado del siguiente modo:

- **Product Owner**: el tutor académico ha asumido de forma aproximada esta función, al validar prioridades, orientar el alcance funcional del prototipo y revisar la utilidad de los entregables desde el punto de vista del objetivo del TFM.
- **Scrum Master**: esta responsabilidad ha sido asumida por el propio autor del trabajo, en la medida en que ha organizado iteraciones, registrado bloqueos, mantenido la trazabilidad del progreso y ajustado la planificación cuando han surgido incidencias técnicas o cambios de enfoque.
- **Developers**: el desarrollo técnico también ha recaído íntegramente en el autor, responsable de la implementación del simulador, la integración de servicios, el entrenamiento de modelos y la documentación de resultados.

En consecuencia, varias responsabilidades propias de un equipo Scrum han quedado concentradas en una única persona, mientras que el tutor ha actuado como agente externo de supervisión, validación y priorización. Esta circunstancia introduce diferencias claras respecto a un SCRUM de equipo, pero no invalida su uso como marco ligero de gestión. En este contexto, lo más relevante no ha sido reproducir todas las ceremonias de forma estricta, sino conservar los elementos que aportan valor real al proyecto: backlog priorizado, trabajo por iteraciones, revisión periódica y registro explícito de decisiones.

La adaptación adoptada se ha basado, por tanto, en los siguientes principios:

- dividir el trabajo en sprints con objetivos acotados y entregables identificables;
- mantener un *Product Backlog* vivo, revisado conforme avanzaba el proyecto;
- registrar en cada iteración qué se planificó, qué se completó y qué problemas aparecieron;
- utilizar las reuniones con el tutor como mecanismo de revisión y re-priorización;
- orientar cada sprint a la obtención de un incremento funcional o documental del proyecto.

En cuanto al seguimiento, las reuniones con el tutor comenzaron a finales de 2025, aproximadamente entre octubre y noviembre, con una periodicidad quincenal, normalmente los martes. Una vez finalizado el primer cuatrimestre y superada la carga principal de exámenes, la cadencia pasó a ser semanal a partir de febrero y marzo de 2026, con el objetivo de acelerar el ritmo de desarrollo y acotar con mayor precisión el trabajo asociado a cada sprint. Este cambio de frecuencia reforzó el carácter iterativo del proyecto y facilitó una validación más cercana del backlog y de los entregables.

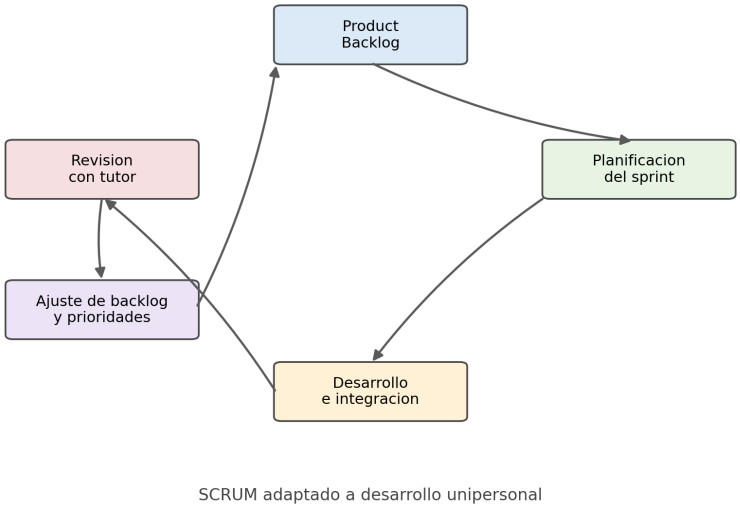

Figura 3.1: Esquema del ciclo de trabajo SCRUM adaptado al desarrollo unipersonal del proyecto. Elaboración propia.

Esta metodología ha encajado bien con la naturaleza del proyecto. Por una parte, ha permitido avanzar de manera incremental desde un prototipo inicial de rutas hasta un MVP funcional con integración de OSRM, OTP, GTFS y un modelo de elección modal. Por otra, ha facilitado documentar de forma ordenada tanto el trabajo ya completado como las líneas de evolución previstas, incluyendo mejoras visuales, ampliación de modelos y posibles análisis adicionales de simulación. A partir de este marco general, en los apartados siguientes se presenta el backlog del proyecto y el histórico de sprints ejecutados.

3.2. Product Backlog

3.2.1. Backlog inicial

El *Product Backlog* se definió como una lista viva de trabajo, organizada en grandes bloques funcionales y técnicos. En lugar de partir de un conjunto cerrado de tareas detalladas desde el inicio, se optó por un backlog de alto nivel que pudiera refinarse a medida que se validaban decisiones de diseño y se consolidaba el MVP del simulador.

En una primera aproximación, los elementos principales del backlog fueron los siguientes:

1. Definir una arquitectura base cliente-servidor para desacoplar frontend, backend y

servicios externos.

2. Construir un prototipo web funcional con mapa interactivo, selección de origen-destino y visualización de rutas.
3. Integrar OSRM en local con perfiles diferenciados para coche, bicicleta y desplazamiento a pie.
4. Incorporar transporte público urbano mediante GTFS de Toledo y OpenTripPlanner.
5. Diseñar una API propia que unifique consultas de rutas, exploración GTFS e inferencia modal.
6. Preparar una pipeline reproducible de datos para el dataset LPMC.
7. Entrenar y validar un primer modelo de elección modal utilizable dentro del simulador.
8. Integrar el modelo de inferencia en backend y frontend de forma modular.
9. Mejorar la interfaz, la experiencia de usuario y la capacidad de análisis visual del sistema.
10. Documentar la solución, registrar la evolución del proyecto y redactar la memoria final.

Conforme avanzó el trabajo, este backlog se fue refinando en tareas más concretas. Algunos elementos inicialmente planteados como trabajo futuro pasaron a formar parte del desarrollo principal, mientras que otros quedaron aplazados para iteraciones posteriores. A fecha de redacción de esta memoria, la parte nuclear del simulador puede considerarse ya implementada, por lo que el backlog actual se concentra principalmente en consolidación documental, mejoras visuales y ampliaciones funcionales sobre una base ya operativa.

Para facilitar su seguimiento, los elementos del backlog se agruparon en cuatro bloques:

- **Bloque funcional**: interfaz web, selección de viajes, cálculo de rutas, visualización de resultados y cuadro de información.
- **Bloque de integración de servicios**: despliegue y conexión de OSRM, OTP y GTFS dentro de una arquitectura reproducible.
- **Bloque de inteligencia artificial**: preprocesado del dataset LPMC, entrenamiento de modelos, validación e integración de inferencia.
- **Bloque documental y de cierre**: trazabilidad del proyecto, redacción de memoria, mejora de la presentación y preparación de resultados.

3.2.2. Priorización de items

La priorización del backlog no se realizó únicamente por orden técnico de implementación, sino por valor aportado al objetivo general del proyecto. En consecuencia, se siguió el siguiente criterio:

- **Prioridad alta**: elementos necesarios para disponer de un simulador funcional extremo a extremo, incluyendo mapa interactivo, rutas por modo, integración de transporte público y una primera versión operativa de la inferencia modal.
- **Prioridad media**: mejoras que aumentan la usabilidad, la claridad visual o la trazabilidad del sistema, pero que no bloquean el funcionamiento básico del MVP.
- **Prioridad evolutiva**: ampliaciones previstas una vez estabilizado el núcleo del sistema, como el entrenamiento de modelos adicionales, la mejora de análisis de escenarios o la incorporación de nuevas visualizaciones y métricas.

Este criterio explica el orden seguido durante el desarrollo. Primero se priorizó la infraestructura mínima para obtener rutas y visualizar resultados reales sobre el mapa; después se incorporó el bloque de modelado e inferencia; y finalmente se abordaron tareas de consolidación, mejora y documentación. De esta forma, cada sprint pudo orientarse a producir incrementos útiles sobre una base ya validada, en lugar de avanzar en paralelo sobre demasiados frentes abiertos.

3.3. Histórico de sprints ejecutados

En este apartado se resume el trabajo realizado en cada sprint. Dado que el proyecto no se inició con una planificación cerrada y que parte de la trazabilidad se ha reconstruido a partir de artefactos, versiones funcionales y reuniones de seguimiento, los periodos temporales y el esfuerzo asociado deben interpretarse como estimaciones razonables. Aun así, este registro resulta útil para mostrar la secuencia real de construcción del sistema, la lógica de priorización seguida y el peso relativo de cada bloque de trabajo.

3.3.1. Sprint 1: Prototipo OSRM remoto

Periodo aproximado: primeros días de diciembre de 2025.
Esfuerzo estimado: 8–12 horas.
Objetivo: validar el flujo mapa $\rightarrow$ backend $\rightarrow$ rutas por modo.
Trabajo realizado:

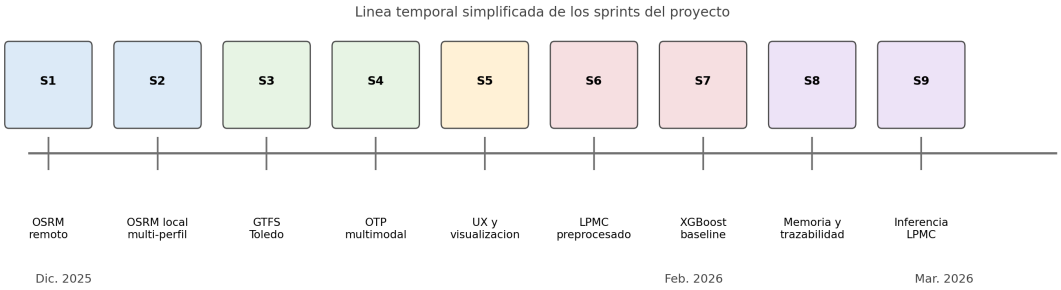

Figura 3.2: Línea temporal simplificada de los sprints ejecutados durante el desarrollo del proyecto. Elaboración propia.

- Arranque del frontend con React + Leaflet.
- Backend FastAPI mínimo para cálculo de rutas.
- Integración inicial con demoserver oficial de OSRM.

Problema detectado: el demoserver devuelve siempre la ruta en coche, lo que no permitía comparación entre `driving`, `cycling` y `foot`.

Entregables: base funcional de la aplicación y validación temprana de arquitectura cliente-servidor.

3.3.2. Sprint 2: OSRM local multi-perfil

Periodo aproximado: primera semana de diciembre de 2025.
Esfuerzo estimado: 12–16 horas.
Objetivo: obtener rutas diferenciadas y control total de infraestructura.
Trabajo realizado:

- Paso intermedio por instancias públicas de `routing.openstreetmap.de`.
- Despliegue final de tres instancias OSRM locales en Docker.
- Configuración de puertos: 5000 (coche), 5001 (bici), 5002 (a pie).
- Consolidación del endpoint `/api/osrm/routes`.

Entregables: comparación modal estable y reproducible en entorno local.

3.3.3. Sprint 3: GTFS Toledo

Periodo aproximado: segunda semana de diciembre de 2025.

Esfuerzo estimado: 10–14 horas.
Objetivo: incorporar red de transporte público urbano en la aplicación.
Trabajo realizado:

- Carga y modelado del feed `GTFS_Urbano_Toledo`.
- Endpoints para paradas, rutas y horarios.
- Visualización de paradas y líneas en el frontend.

Entregables: capa GTFS operativa para exploración de red y horarios.

3.3.4. Sprint 4: OTP multimodal

Periodo aproximado: entre diciembre de 2025 y febrero de 2026.
Esfuerzo estimado: 16–24 horas.
Objetivo: habilitar rutas puerta a puerta con transporte público.
Trabajo realizado:

- Construcción del `graph.obj` con OSM + GTFS.
- Endpoint `/api/otp/routes` con ordenación y paginación de itinerarios.
- Segmentación de legs (WALK/BUS) y exposición de metadatos de línea.

Entregables: integración completa de modo tránsito con desglose por segmentos.

3.3.5. Sprint 5: Mejora visual y UX

Periodo aproximado: en paralelo a la consolidación de GTFS y OTP.
Esfuerzo estimado: 8–12 horas.
Objetivo: hacer más clara la comparación entre modos.
Trabajo realizado:

- Estilos diferenciados por modo (colores y trazos).
- Visualización de segmentos OTP solo en modo tránsito.
- Mejoras de panel de información y selección de líneas.

Entregables: interfaz más legible y usable para análisis de escenarios.

3.3.6. Sprint 6: LPMC exploración y preprocesado

Periodo aproximado: segunda mitad de diciembre de 2025.

Esfuerzo estimado: 12–18 horas.

Objetivo: investigar el dataset LPMC y los trabajos de referencia (papers adjuntos) para preparar una base sólida de modelado.

Trabajo realizado:

- Revisión técnica del dataset LPMC y de los papers de referencia incluidos en la carpeta `papers`.
- Scripts de exploración y preprocesado del dataset LPMC.
- Codificación de variables categóricas y limpieza de columnas.
- Split temporal train/test por `survey_year`.

Entregables: datasets preprocesados reproducibles para entrenamiento de modelos.

3.3.7. Sprint 7: XGBoost baseline

Periodo aproximado: entre finales de diciembre de 2025 y febrero de 2026.

Esfuerzo estimado: 14–20 horas.

Objetivo: entrenar un primer modelo útil y evaluable.

Trabajo realizado:

- Entrenamiento de XGBoost multiclase.
- Escalado de variables numéricas y persistencia de artefactos.
- Validación por métricas de test e inspección de inferencias.

Entregables: `xgb_lpmc_baseline.joblib`, `xgb_lpmc_scaler.joblib` y resultados base para comparación.

3.3.8. Sprint 8: Arranque de memoria y trazabilidad SCRUM

Periodo aproximado: primera quincena de marzo de 2026.

Esfuerzo estimado: 10–14 horas.

Objetivo: iniciar la redacción formal de la memoria y consolidar el registro estructurado del trabajo realizado.

Trabajo realizado:

- Inicio de la escritura de la memoria en LaTeX con estructura completa por capítulos.
- Propuesta y consolidación del índice actual del TFM.
- Registro del trabajo acumulado en formato SCRUM para seguimiento con tutor.
- Redacción inicial del capítulo de estado del arte.

Entregables: base de memoria preparada para redacción iterativa y seguimiento SCRUM trazable.

3.3.9. Sprint 9: Integración de inferencia LPMC en simulador

Periodo aproximado: segunda mitad de febrero de 2026.

Esfuerzo estimado: 14–20 horas.

Objetivo: conectar el modelo de elección modal con backend y frontend para inferencia operativa por escenario.

Trabajo realizado:

- Implementación de endpoint `/api/lpmc/predict` en FastAPI.
- Implementación de endpoint `/api/lpmc/debug-features` para inspección del vector de entrada al modelo.
- Integración en frontend de formulario de perfil usuario y visualización de probabilidades por modo.
- Ajuste de la estrategia de `household_id`: reentrenamiento de una variante de modelo sin esta variable como feature (`nohh`), usándola solo para agrupación de folds.
- Conservación de artefactos legacy (con `household_id`) como respaldo y conmutación por variable de entorno (`LPMC_MODEL_VARIANT`).
- Correcciones de interfaz (castellano, orden de bloques y legibilidad de resultados).

Entregables: flujo completo de inferencia (entrada de variables, cálculo de features de ruta, predicción y depuración de features).

3.4. Sprint reviews y retrospectivas

3.4.1. Aprendizajes por iteración

- Validar pronto las limitaciones de servicios externos evita retrabajo (caso demoserver OSRM).

- Mantener infraestructura local mejora reproducibilidad y control experimental.
- Separar integración funcional y capa de IA reduce complejidad y facilita depuración.
- Mantener trazabilidad escrita del proyecto es clave para retomar el trabajo con rapidez tras varias semanas de pausa.
- Exponer endpoints de depuración acelera la validación del pipeline de inferencia en integración.

3.4.2. Acciones de mejora

- Integrar inferencia en backend sin bloquear el servidor.
- Sustituir parámetros temporales hardcodeados de OTP por fecha/hora seleccionadas por usuario.
- Revisar la coherencia entre oferta real de transporte público y espacio de decisión del modelo: en trayectos cortos donde OTP no devuelve tramo BUS (itinerario íntegramente WALK), no debe habilitarse la alternativa `pt` en inferencia.
- Añadir trazabilidad explícita entre evidencias (scripts, notebooks, endpoints) y sprints.
- Estandarizar formato de revisión al cierre de cada iteración.

3.5. Seguimiento operativo

3.5.1. Registro en SCRUM_LOG

El seguimiento detallado se mantiene en el documento vivo `latex/SCRUM_LOG.md`, donde se registran:

- estado de cada sprint,
- bloqueos,
- evidencia técnica,
- decisiones tomadas y próximos pasos.

3.5.2. Trazabilidad con el repositorio

Cada sprint debe poder vincularse con evidencias concretas:

- archivos modificados,
- scripts ejecutables,
- artefactos de modelo,
- endpoints funcionales,
- resultados observables en interfaz.

3.6. Próximo sprint propuesto

3.6.1. Objetivo

Actualizar OTP y datos de transporte para 2026, eliminando el parche temporal de fecha/hora fija y utilizando fecha/hora de viaje seleccionadas por el usuario.

3.6.2. Alcance y criterio de cierre

- Feed GTFS actualizado (2026) y `graph.obj` regenerado.
- Selector de fecha y hora de viaje integrado en frontend.
- Paso de `date`/`time` desde UI hasta `_build_otp_params` en backend.
- Comprobación de itinerarios OTP con fecha/hora variable en Toledo.
- Evidencia registrada en `SCRUM_LOG.md` y en la memoria.

3.6.3. Plan a dos sprints vista

La línea de investigación DNN (Keras/TensorFlow en CPU, ajuste de depth/width y dropout) se planifica para dos sprints posteriores al de actualización OTP, una vez estabilizado el flujo de rutas con fecha/hora seleccionables. En paralelo, se propone una tarea de robustez metodológica en inferencia modal: restringir el conjunto de alternativas a modos realmente disponibles en cada consulta. En particular, si el planificador no devuelve transporte público efectivo (sin tramo BUS), la elección debe realizarse solo entre `walk`, `cycle` y `drive`, evitando clasificar como `pt` desplazamientos que operativamente se resuelven andando.

4. Arquitectura/Diseño de la solución

4.1. Arquitectura general del sistema

4.2. Diseño del backend

4.3. Diseño del frontend

4.4. Integración OSRM, OTP y GTFS

4.5. Diseño de integración del modelo IA

4.6. Decisiones técnicas y trade-offs

5. Implementación y resultados

5.1. Implementación de OSRM local

5.1.1. Preprocesado por perfiles

Car, bike y foot

5.1.2. Despliegue de contenedores

Puertos y verificación

5.2. Implementación de OTP con Toledo

5.2.1. Construcción de grafo

Entradas OSM y GTFS

5.2.2. Explotación del planificador

Parámetros relevantes

5.3. Implementación del backend

5.3.1. Endpoints y servicios

OSRM, OTP y GTFS

5.4. Implementación del frontend

5.4.1. Interfaz y mapa interactivo

Flujos de usuario

5.5. Integración de datos GTFS

5.5.1. Carga y procesamiento

Rutas, paradas y horarios

5.6. Entrenamiento de los modelos de aprendizaje automático

5.6.1. Descripción del dataset

LPMC

5.6.2. Preprocesado de datos

5.6.3. Ajuste de hiperparámetros

5.6.4. Entrenamiento y evaluación

6. Conclusiones y trabajo futuro

6.1. Conclusiones técnicas

6.2. Cumplimiento de objetivos

6.3. Trabajo futuro

A. Anexos

A.1. Manual de despliegue y ejecución

A.1.1. Requisitos del entorno

Versiones recomendadas

A.1.2. Arranque de servicios

OSRM, OTP, backend y frontend

A.2. Endpoints y contratos API

A.2.1. Contratos del backend

Ejemplos de petición y respuesta

A.3. Tablas de hiperparámetros y resultados completos

A.3.1. Configuraciones experimentales

Resultados detallados por experimento

A.4. Evidencias de sprints

A.4.1. Registro de tareas y decisiones

Referencias a commits y artefactos

Referencia bibliográfica

[Breiman, 2001] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

[Chen and Guestrin, 2016] Chen, T. and Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

[Delling et al., 2017] Delling, D., Goldberg, A. V., Pajor, T., and Werneck, R. F. (2017). Customizable route planning in road networks. *ACM Journal of Experimental Algorithmics*, 22:1–48.

[EMT Valencia, 2026] EMT Valencia (2026). Nap transporte multimodal - fichero de detalle 965 (autobus urbano de valencia). `https://nap.transportes.gob.es/Files/Detail/965`. Consultado el 23 de febrero de 2026.

[Friedman, 2001] Friedman, J. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29:1189–1232.

[Geisberger et al., 2008] Geisberger, R., Sanders, P., Schultes, D., and Delling, D. (2008). Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Proceedings of the 7th International Conference on Experimental Algorithms (WEA)*, pages 319–333.

[Goodfellow et al., 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

[Google, 2026a] Google (2026a). Google maps platform billing and pricing. `https://developers.google.com/maps/billing-and-pricing/pricing`. Consultado el 22 de febrero de 2026.

[Google, 2026b] Google (2026b). Routes api overview. https://developers.google.com/maps/documentation/routes/overview. Consultado el 22 de febrero de 2026.

[Haklay, 2010] Haklay, M. (2010). How good is volunteered geographical information? a comparative study of openstreetmap and ordnance survey datasets. *Environment and Planning B: Planning and Design*, 37(4):682–703.

[HeiGIT gGmbH, 2026] HeiGIT gGmbH (2026). openrouteservice api documentation. https://giscience.github.io/openrouteservice/api-reference/. Consultado el 23 de febrero de 2026.

[Hillel, 2019] Hillel, T. (2019). London passenger mode choice (lpmc) – dataset description. Documento tecnico del dataset (archivo local: papers/CS_LPMC.pdf). Consultado el 22 de febrero de 2026.

[Hillel et al., 2018] Hillel, T., Elshafie, M. Z. E. B., and Jin, Y. (2018). Recreating passenger mode choice-sets for transport simulation: A case study of london, uk. *Proceedings of the Institution of Civil Engineers - Smart Infrastructure and Construction*, 171(1):29–42.

[Horni et al., 2016] Horni, A., Nagel, K., and Axhausen, K. W., editors (2016). *The Multi-Agent Transport Simulation MATSim*. Ubiquity Press.

[José Ángel Martín-Baos, 2023] José Ángel Martín-Baos, Ricardo García Rodenas, L. R. B. (2023). A prediction and behavioural analysis of machine learning methods for modelling travel mode choice. *Transportation Research Part C: Emerging Technologies*, 156:104318.

[Lopez et al., 2018] Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Fl"otter"od, Y.-P., Hilbrich, R., L"ucken, L., Rummel, J., Wagner, P., and Wie fner, E. (2018). Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, pages 2575–2582.

[Mapbox, 2026] Mapbox (2026). Directions api. https://docs.mapbox.com/api/navigation/directions/. Consultado el 23 de febrero de 2026.

[Martín-Baos, 2023] Martín-Baos, J. A. (2023). *Machine Learning Methods Applied to Transport Demand Modelling*. Phd thesis, University of Castilla-La Mancha. Tesis doctoral.

[McFadden, 1974] McFadden, D. (1974). Conditional logit analysis of qualitative choice behavior. In Zarembka, P., editor, *Frontiers in Econometrics*, pages 105–142. Academic Press.

[MobilityData, 2026a] MobilityData (2026a). Gtfs schedule best practices. https://gtfs.org/documentation/schedule/schedule-best-practices/. Consultado el 23 de febrero de 2026.

[MobilityData, 2026b] MobilityData (2026b). Gtfs schedule reference. https://gtfs.org/documentation/schedule/reference/. Consultado el 23 de febrero de 2026.

[MobilityData, 2026c] MobilityData (2026c). Gtfs validator (github repository). https://github.com/MobilityData/gtfs-validator. Consultado el 23 de febrero de 2026.

[NAP Transporte Multimodal, 2026a] NAP Transporte Multimodal (2026a). Nap transporte multimodal - home. https://nap.transportes.gob.es/. Consultado el 23 de febrero de 2026.

[NAP Transporte Multimodal, 2026b] NAP Transporte Multimodal (2026b). Nap transporte multimodal - quienes somos. https://nap.transportes.gob.es/quienes-somos. Consultado el 23 de febrero de 2026.

[openrouteservice, 2026] openrouteservice (2026). Api restrictions and limits. https://openrouteservice.org/restrictions/. Consultado el 23 de febrero de 2026.

[OpenTripPlanner Team, 2026a] OpenTripPlanner Team (2026a). Build configuration (opentripplanner documentation). https://docs.opentripplanner.org/en/latest/BuildConfiguration/. Consultado el 23 de febrero de 2026.

[OpenTripPlanner Team, 2026b] OpenTripPlanner Team (2026b). Opentripplanner documentation. https://docs.opentripplanner.org/en/latest/. Consultado el 23 de febrero de 2026.

[OpenTripPlanner Team, 2026c] OpenTripPlanner Team (2026c). Opentripplanner (github repository). https://github.com/opentripplanner/OpenTripPlanner. Consultado el 23 de febrero de 2026.

[OpenTripPlanner Team, 2026d] OpenTripPlanner Team (2026d). Routerequest (opentripplanner documentation). https://docs.opentripplanner.org/en/latest/RouteRequest/. Consultado el 23 de febrero de 2026.

[OSM Foundation, 2026] OSM Foundation (2026). About openstreetmap. https://www.openstreetmap.org/about. Consultado el 23 de febrero de 2026.

[Project OSRM, 2026a] Project OSRM (2026a). Osrm api documentation v5.24.0. https://project-osrm.org/docs/v5.24.0/api/. Consultado el 23 de febrero de 2026.

[Project OSRM, 2026b] Project OSRM (2026b). osrm-backend (github repository). https://github.com/Project-OSRM/osrm-backend. Consultado el 23 de febrero de 2026.

[Schwaber and Sutherland, 2020] Schwaber, K. and Sutherland, J. (2020). The scrum guide: The definitive guide to scrum: The rules of the game. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf. Consultado el 17 de marzo de 2026.

[TomTom, 2026] TomTom (2026). Routing api - calculate route. https://developer.tomtom.com/routing-api/documentation/tomtom-maps/calculate-route. Consultado el 23 de febrero de 2026.

[Train, 2009] Train, K. E. (2009). *Discrete Choice Methods with Simulation*. Cambridge University Press, 2 edition.

[Unauto, 2026] Unauto (2026). Nap transporte multimodal - fichero de detalle 1377 (autobuses urbanos toledo). https://nap.transportes.gob.es/Files/Detail/1377. Consultado el 23 de febrero de 2026.